Project Report

Smart Medicine

Presented By

Teerapat Kotanart 6604062636348

Phongsiri Loedphongthai 6604062636453

Present to

Associate Professor Kobkiat Saraubon, Ph.D.

This report is part of the Mobile Application Development subject.

Junior 2 2026

King Mongkut's University of Technology North Bangkok

Contents

Section 1: Overview 1
Section 2: System Architecture Diagram 2
Section 3: UX/UI screen flow 3
Section 4: Data Structure 4
Section 5: Navigation Bar 5
Section 6: Search / Filter 6
Section 7: Login / Registration 7
Section 8: Save login Status / Token (SharedPreferences) 8
Section 9: Security 9
Section 10: Master-Detail 10
Section 11: Backend / API 14
Section 12: database, Join, SQL 15
Section 13: CRUD (DB) 17

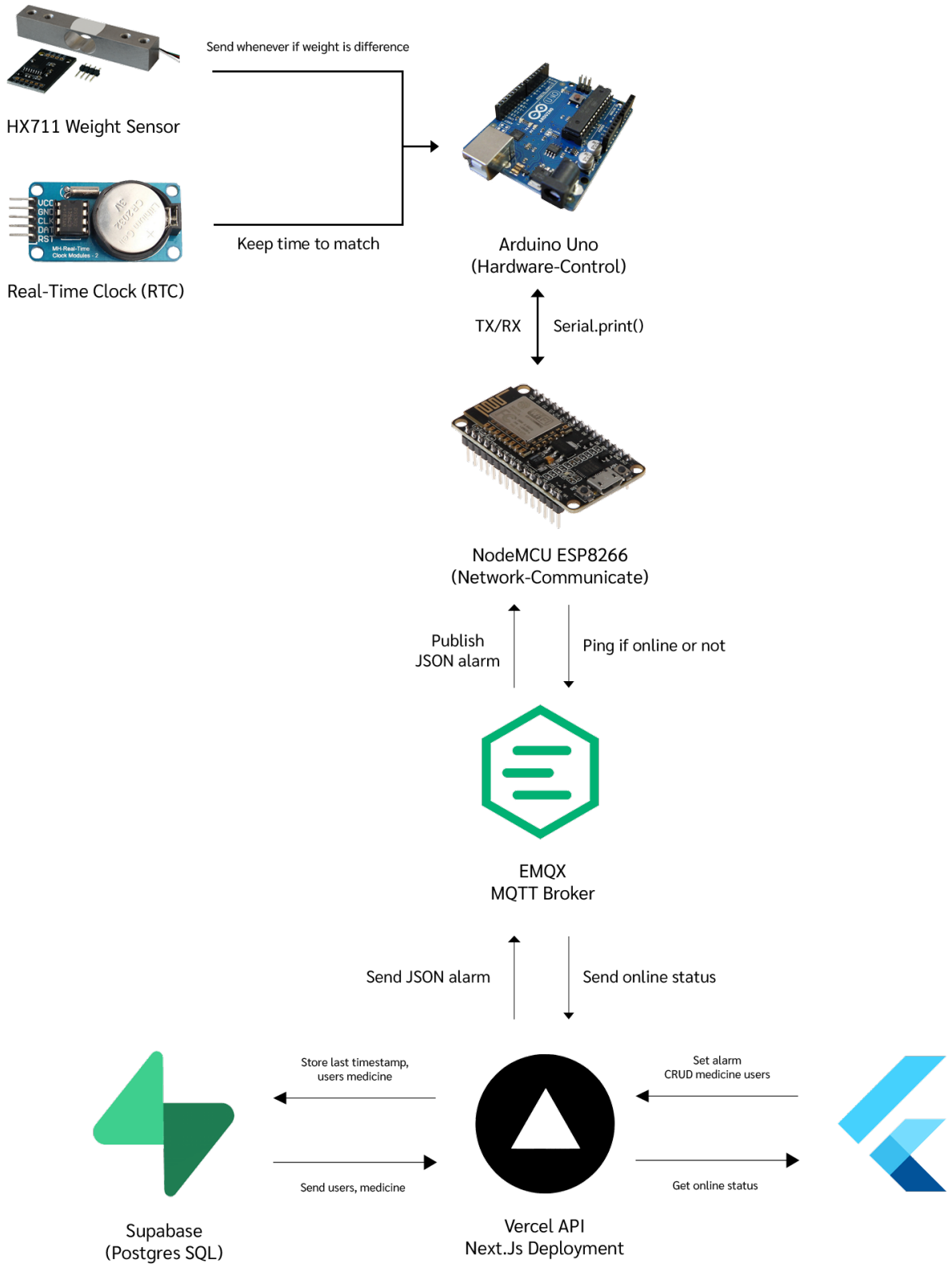
Section 1: Overview

This smart medicine application is designed for patient and can manage their daily medication schedules. If who someone have a patient and them forget to take their medicine this application can manage and arrange the drugs for patient. This application can check your patient eat drugs yet and can add the drugs set the time for each drug.

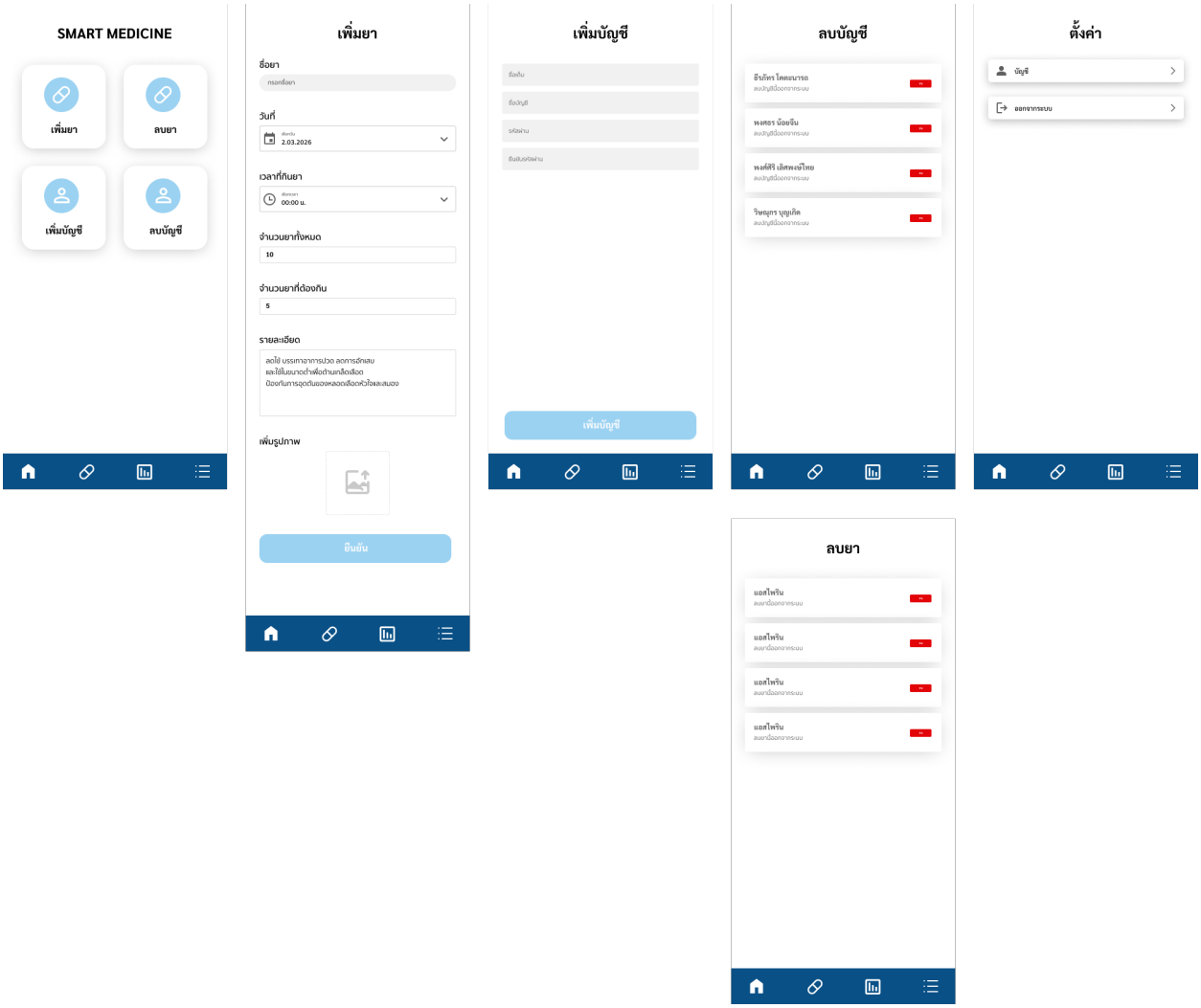
The system of a mobile application development using flutter, backend API service, and smart medicine box device connected through MQTT, can add, update, or remove the drug through mobile application, additionally this app can follow the schedules drug of patients what have how many drug patients must to eat.

Conclude, the smart medicine improves medication adherence, reduce the risk of missed doses, and provide a convenient monitoring tool for caregivers.

Section 2: System Architecture Diagram



Section 3: UX/UI screen flow



Section 4: Data Structure

This structure stores medication information including dosage, timing, and medication schedule.

Example JSON

```
    {
        "drug_id": 32,
        "drug_name": "Golden",
        "total_drugs": 10,
        "each_taken": 2,
        "timing": "before",
        "frequency": "daily"
    }
```

Alarm Schedule Data structure

This alarm schedule structure is used to send medication reminder times to the smart medication box using MQTT communication.

Field	Type	Description
username	String	Patient username
alarms	Array	List of medication alarm schedules

Example JSON

```
{
  "username": "ahgong",
  "alarms": [
    {
      "name": "Golden",
      "time": "08:00"
    },
    {
      "name": "Gofen",
      "time": "12:00"
    }
  ]
}
```

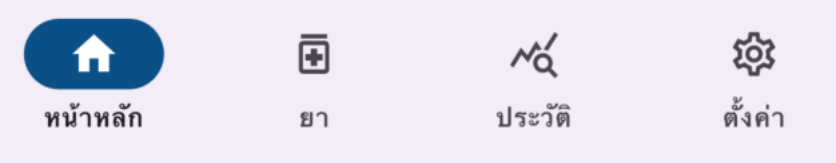
This structure allows the system to send multiple medication alarms to the smart medicine box.

Section 5: Navigation Bar

We user **navigation destination** in this application, it so clean, modern and easy to use

```
class _NavigationState extends State<Navigation> {
  int _selectedIndex = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      bottomNavigationBar: NavigationBar(
        selectedIndex: _selectedIndex,
        onDestinationSelected: (int index) {
          setState(() {
            _selectedIndex = index;
          });
        },
        indicatorColor: const Color(0xFF095086),
        destinations: const <Widget>[
          NavigationDestination(
            selectedIcon: Icon(Icons.home, color: Colors.white),
            icon: Icon(Icons.home_outlined),
            label: 'หน้าหลัก',
          )
        ],
      ),
      body: [
        HomePage(user: widget.user),
      ][_selectedIndex],
    );
  }
}
```



Section 6: Search / Filter

The search input field allows users to enter a keyword to search for medications. When the user types in the search box, the onChanged event triggers the search function.

```
                  child: TextField(
                    controller: searchController,
                    decoration: const InputDecoration(
                      hintText: "ค้นหายา",
                      prefixIcon: Icon(Icons.search),
                      border: OutlineInputBorder(),
                    ),
                    onChanged: searchDrug,
                  ),
```

Search Filtering Function

The search function filters the medication list based on the keyword entered by the user.

```
void searchDrug(String keyword) {
  setState(() {
    if (keyword.isEmpty) {
      filteredUsers = allUsers;
    } else {
      filteredUsers = allUsers.map((user) {
        var drugs = List.from(user["drugs"]).where((drug) {
          return drug["drug_name"].toLowerCase().contains(
            keyword.toLowerCase(),
          );
        }).toList();

        return {"username": user["username"], "drugs": drugs};
      }).toList();
    }
  });
}
```

Section 7: Login / Registration

In this app we can't create new account for admin or register for patient. But if you want to add patient you must login admin account we created and use "add patient widget"

```
final response = await http.post(
  Uri.parse('https://smart-medicine-topaz.vercel.app/api/login'),
  headers: {'Content-Type': 'application/json; charset=UTF-8'},
  body: jsonEncode({
    'username': _usernameController.text,
    'password': _passwordController.text,
  }),
);
```

This code will send username and password to API and check username from database.

```
If (role == 'admin') {
  Navigator.of(context).pushReplacement(
    MaterialPageRoute(builder: (context) => LandingPage(user: user)),
  );
} else if (role == 'elderly') {
  Navigator.of(context).pushReplacement(
    MaterialPageRoute(
      builder: (context) => LandingPagePatient(user: user),
    ),
  );
} else {
  _showErrorDialog('Unauthorized role: $role');
}
```

And this code will check role from account you login.

```
TextField(
  controller: _usernameController,
  decoration: InputDecoration(
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(12),
    ),
    contentPadding: const EdgeInsets.symmetric(horizontal: 16),
  ),
),
```

This is example code input box for input username and password

Section 8: Save login Status / Token (SharedPreferences)

When we login application will fetch and save **token** you login

POST/API

```
https://smart-medicine-topaz.vercel.app/api/login
```

Raw

```
{
    “username”: “xobazjr”,
    “password”: “admin”
}
```

Body


```
{
    “message”: “Login successful”,
    “token”:
“eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI0ZDMwYWQ0Ny03ZTMyLTRhMmQtYTI1My0y
OTFlMzlmNzhiOTAiLCJ1c2VybmFtZSI6InhvYmF6anIiLCJyb2xlIjoiYWRtaW4iLCJpYXQiOjE3NzMwNzQxMj
gsImV4cCI6MTc3NTY2NjEyOH0.8sQ78i11A0RVAWGkG859krrVZzVuE_6TcWKm7herlMs”,
    “user”: {
        “user_id”: “4d30ad47-7e32-4a2d-a253-291e39f78b90”,
        “username”: “xobazjr”,
        “role”: “admin”,
        “tel”: “0954849802”
    }
}
```


And then we got the token. Token will contain to **SharedPreferences**

```
await prefs.setString(‘token’, data[‘token’]);
```

And this token can use with other pages e.g., add/ delete/ view drugs

```
    final mqttRes = await http.post(
      Uri.parse(“https://smart-medicine-topaz.vercel.app/api/mqtt/set_box”),
      headers: {
        “Authorization”: “Bearer $token”,
        “Content-Type”: “application/json”,
      },
      body: jsonEncode({“username”: “ahgong”, “alarms”: alarms}),
```

Section 9: Security

From section 9 we use token to verify when user login is correct or not, if yes server will create token turn back to you


```
"token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI0ZDMwYWQ0Ny03ZTMyLTRhMmQtYTI1My0y
OTFlMzlmNzhiOTAiLCJ1c2VybmFtZSI6InhvYmF6anIiLCJyb2xlIjoiYWRtaW4iLCJpYXQiOjE3NzMwNzQxMj
gsImV4cCI6MTc3NTY2NjEyOH0.8sQ78i11A0RVAWGkG859krrVZzVuE_6TcWKm7herlMs",
```


And all page in this application must to send token always e.g.,

```
https://smart-medicine-topaz.vercel.app/api/medicine/list?caretaker_name=xobazjr
```

Headers

Key	Value
Authorization	Bearer "Insert token"

Body

```
{
    "username": "ahguay",
    "drugs": []
}
```

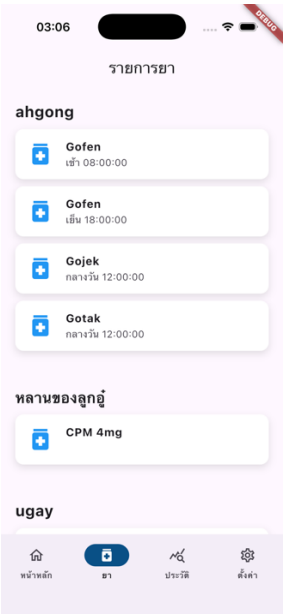
And if token equal null

```
{
    "error": "Unauthorized: Missing or invalid token"
}
```

Section 10: Master-Detail

The medication list page is implemented using a ListView widget which represents the master view. When user selects a medication item.

Master View



Show list medicine e.g.,

Paracetamol

Golden

Gofen

Example code

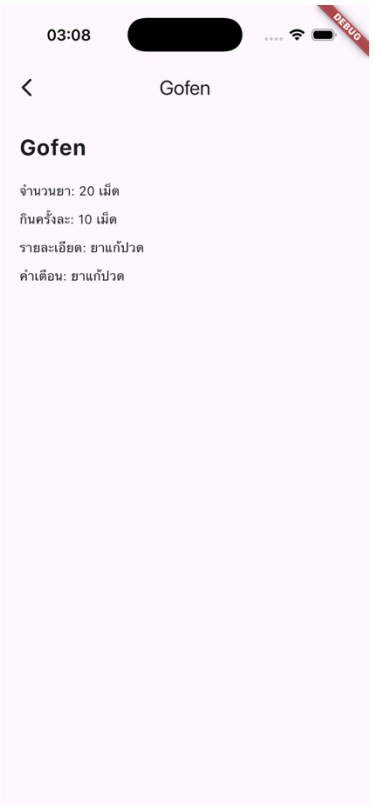
```
ListView(
  children: users.map((user) {
    final drugs = user["drugs"]
        .where((drug) => drug["total_drugs"] != 0)
        .toList();

    return Column(
      children: drugs.map<Widget>((drug) {
        return ListTile(
          leading: Icon(Icons.medication),
          title: Text(drug["drug_name"]),
          subtitle: Text(getTimeText(drug)),

          onTap: () {
            Navigator.push(
              context,
              MaterialPageRoute(
                builder: (context) => DrugDetailPage(drug: drug),
              ),
            );
          },
        );
      }).toList(),
    );
  }).toList(),
)
```

The master view displays a list of medications using the ListView widget. Each item in the list represent a medication entry. When a user selects a medication, the application navigation to the detail view to display more information about the selected drug

Detail View



And when you click some drug it view show detail of that drug e.g.,

```
จำนวนยา
กินครั้งละ
รายละเอียด
คำเตือน
```

Example code

```
class DrugDetailPage extends StatelessWidget {
  final Map<String, dynamic> drug;

  const DrugDetailPage({super.key, required this.drug});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(drug["drug_name"]),
      ),
      body: Padding(
        padding: EdgeInsets.all(20),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [

            Text(
              drug["drug_name"],
              style: TextStyle(
                fontSize: 24,
                fontWeight: FontWeight.bold,
              ),
            ),

            SizedBox(height: 20),

            Text("Total Drugs: ${drug["total_drugs"]}"),
            Text("Each Taken: ${drug["each_taken"]}"),
            Text("Description: ${drug["description"]}"),
            Text("Warning: ${drug["warning"]}"),

          ],
        ),
      ),
    );
  }
}
```

The detail view displays detailed information about the selected medication. This includes the drug name, dosage, description, and warning information. The detail page is opened when a user selects an item from the master list.

Section 11: Backend / API

The system uses token to ensure secure access to the API. When a user logs in successfully, the server generates a token and returns it to the mobile application.

The token must be included in the **Authorization header** for subsequent API requests.

Example:

```
Authorization: Bearer “TOKEN”
```

Login API Example

This API is used for user authentication

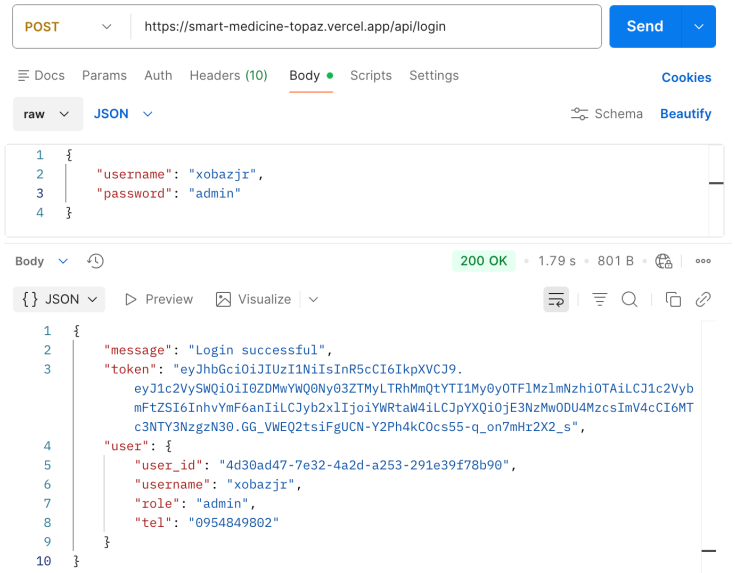
```
POST /api/login
```

Request Example

```
{
    “username”: “xobazjr”,
    “password”: “admin”
}
```

Response Example


```
“token”:
“eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI0ZDMwYWQ0Ny03ZTMyLTRhMmQtYTI1My0y
OTFlMzlmNzhiOTAiLCJ1c2VybmFtZSI6InhvYmF6anIiLCJyb2xlIjoiYWRtaW4iLCJpYXQiOjE3NzMwODU4Mz
csImV4cCI6MTc3NTY3NzgzN30.GG_VWEQ2tsiFgUCN-Y2Ph4kCOcs55-q_on7mHr2X2_s”,
```

Add Medicine API

This API allows caregivers to add a new medication schedule

POST /api/login

Example request

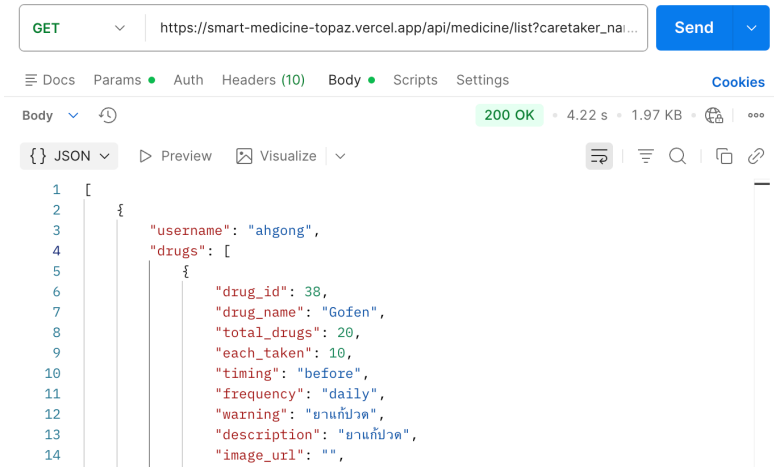
```
{
  “drug_name”: “Paracetamol”,
  “start_date”: “2026-03-10T08:15:30.000Z”,
  “start_time”: “08:00”,
  “total_drugs”: 10,
  “each_taken”: 2,
  “description”: “กินแก้ปวด”,
  “warning”: “กินแก้ปวด”,
  “image_url”: “”,
  “timing”: “after”,
  “frequency”: “daily”,
  “username”: “ahgong”,
  “take_morning”: true,
  “take_noon”: false,
  “take_evening”: true,
  “take_bedtime”: false
}
```

Get Medicine List API

This API retrieves medication data for a specific user

GET /api/medicine/list

Example response



Section 12: Database, Join, SQL

Example Database Tables

users

Field	Type	Description
user_id	UUID	Unique user identifier
username	VARCHAR	Username
password_hash	VARCHAR	Encrypted password
role	VARCHAR	User role
created_at	TIMESTAMP	Creation time

Drugs

Field	Type	Description
drug_id	INT	Unique drug identifier
drug_name	VARCHAR	Name of medication
total_drugs	INT	Total number of pills
each_taken	INT	Number of pills per dose
timing	VARCHAR	Before or after meals
frequency	VARCHAR	Frequency of medication
user_id	UUID	Patient ID

SQL Example: Insert Data

Example SQL query used to insert a new medication into the database

```sql
INSERT INTO drugs (
  drug_name,
  total_drugs,
  each_taken,
  timing,
  frequency,
  user_id
)
VALUES (
  'Paracetamol',
  20,
  1,
  'after',
  'daily',
  '73850816-772f-48fe-af97-e3c21bc48f45'
);
```

This query stores a new medication record in the database for a specific patient.

SQL Example: SELECT Query

The following SQL query retrieves medication data from the database.

```sql
SELECT *
FROM drugs
WHERE user_id = '73850816-772f-48fe-af97-e3c21bc48f45';
```

This query returns all medications belonging to a specific patient.

SQL JOIN Example

The system user JOIN operations to combine information from mutiple tables

```sql
SELECT users.username, drugs.drug_name, drugs.total_drugs
FROM drugs
JOIN users ON drugs.user_id = users.user_id;
```

This query combines data from the users and drugs tables to display medication information together with patient username

Section 13: CRUD (DB)

The smart midecine application implements CRUD operations (Create, Read, Update, Delete) to manage medication data stored in the database.

Create (Add Medication)

Example implementation

```
Future<void> addDrug(Map<String, dynamic> drugData) async {
  final response = await http.post(
    Uri.parse("https://api/medicine/add"),
    headers: {"Content-Type": "application/json"},
    body: jsonEncode(drugData),
  );
}
```

Read (Retrieve Medication Data)

```
Future<List<dynamic>> fetchDrugs() async {
  final response = await http.get(
    Uri.parse("https://api/medicine/list"),
  );

  return jsonDecode(response.body);
}
```

Update (Edit Medication Information)

```
Future<void> updateDrug(int id, Map<String, dynamic> drugData) async {
  await http.put(
    Uri.parse("https://api/medicine/update/$id"),
    headers: {"Content-Type": "application/json"},
    body: jsonEncode(drugData),
  );
}
```

Delete (Remove Medication)

```
Future<void> deleteDrug(int id) async {
  await http.delete(
    Uri.parse("https://api/medicine/delete/$id"),
  );
}
```

The CRUD operation enables efficient management of medication data in the Smart Medicine system. By integrating RESTful APIs with Flutter application